

SQL SERVER 2022 • AVANZADO

Diagnóstico de queries lentas y timeouts en sistemas legacy (I)

Sesión 3 de 6 • Planes de ejecución complejos, estimación de cardinalidad, conversiones implícitas, spills y lookups

Anatomía de un timeout

- El timeout lo decide el cliente, no SQL Server: 30 s por defecto en ADO.NET, 0 (infinito) en SSMS
- Al vencer, el cliente envía una señal de cancelación: la consulta se aborta y aparece el evento Attention
- La transacción abierta no se deshace sola si la aplicación no gestiona el error: bloqueos huérfanos
- Causas: plan malo, bloqueo, espera de memoria (RESOURCE_SEMAPHORE), compilación costosa o I/O
- Por eso "en SSMS va rápido" no demuestra nada: otro plan, otras opciones SET, sin límite de tiempo

La trampa de ARITHABORT

SSMS usa SET ARITHABORT ON y ADO.NET OFF. Como es una opción que forma parte de la clave de la caché, cada uno recibe su propio plan. Para reproducir lo que ve la aplicación, igualad las opciones SET.

01

Leer un plan de ejecución como un profesional

Tipos de plan, cómo obtenerlos, en qué orden leerlos y qué propiedades importan

Plan estimado, real y en vivo

Plan estimado

- Se genera sin ejecutar: seguro en producción
- Solo estimaciones: filas, costes, memoria
- Es lo que hay en la caché de planes
- Útil para consultas que no terminan nunca
- No revela errores de estimación

Plan real y en vivo

- Real: tras ejecutar, con filas reales, ejecuciones, I/O y spills
- En vivo: Live Query Statistics mientras corre
- Lightweight profiling activo por defecto desde 2019
- `LAST_QUERY_PLAN_STATS`: último plan real sin reejecutar
- Comparar estimado frente a real es el 80 % del diagnóstico

Método de lectura en cinco pasadas

1

Operador raíz

Propiedades del SELECT: tiempo, memoria, avisos, opciones SET, parámetros compilados

2

Avisos

Triángulos amarillos: conversiones, spills, sin predicado de join, concesión excesiva

3

Flechas gruesas

Dónde se mueven muchas filas; comparar real frente a estimado

4

Operadores caros

Por tiempo real (no por % de coste estimado) y por ejecuciones

5

Accesos a datos

Scans, seeks, lookups y predicados residuales en cada hoja

El porcentaje de coste que muestra el plan es siempre una estimación, incluso en el plan real.

Herramientas para capturar el plan correcto

```
-- Medición en la sesión
SET STATISTICS IO, TIME ON;           -- lecturas lógicas/físicas y CPU por sentencia
SET STATISTICS XML ON;                -- plan real como resultado adicional

-- Último plan real desde la caché (sin reejecutar)
ALTER DATABASE SCOPED CONFIGURATION SET LAST_QUERY_PLAN_STATS = ON;
SELECT TOP (10) qs.execution_count, qs.total_worker_time/1000 AS cpu_ms,
               st.text, qps.query_plan AS ultimo_plan_real
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan_stats(qs.plan_handle) qps
ORDER BY qs.total_worker_time DESC;

-- Plan en curso de una sesión que está tardando
SELECT * FROM sys.dm_exec_query_statistics_xml(<session_id>);
```

Estimación de cardinalidad: la raíz de casi todo

- Estadísticas: histograma de hasta 200 pasos sobre la primera columna, densidad y número de filas
- Actualización automática por umbral dinámico ($\approx$ raíz de $1000 \times$ filas) y muestreo en tablas grandes
- Problema de clave ascendente: las filas recientes quedan fuera del histograma
- Predicados correlacionados, funciones sobre columnas y variables locales degradan la estimación
- CE feedback (2022) corrige supuestos de correlación y de fila objetivo en consultas repetitivas

Diagnóstico

Buscad el primer operador, empezando por la derecha, donde real y estimado difieren en más de un orden de magnitud. Todo lo que está a su izquierda hereda el error.

Demo en vivo · Leer un histograma

```
-- ¿Qué estadísticas tiene Pedidos y cuándo se actualizaron?
SELECT s.name, s.auto_created, sp.last_updated, sp.rows, sp.rows_sampled,
       sp.modification_counter
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE s.object_id = OBJECT_ID('dbo.Pedidos');

-- Histograma de ClienteID
DBCC SHOW_STATISTICS ('dbo.Pedidos', 'IX_Pedidos_ClienteID') WITH HISTOGRAM;

-- Misma información en formato consultable
SELECT step_number, range_high_key, equal_rows, range_rows, average_range_rows
FROM sys.dm_db_stats_histogram(OBJECT_ID('dbo.Pedidos'), 2);
```

Demostración del instructor (10 min). Los alumnos la repiten al inicio del Lab 5A.

02

Operadores y patrones costosos

Conversiones implícitas, spills a tempdb, Key/RID Lookups y funciones escalares

Conversiones implícitas

- Si los tipos no coinciden, SQL Server convierte el de menor precedencia al de mayor (VARCHAR → NVARCHAR)
- Si la conversión cae sobre la columna, el predicado deja de ser SARGable: Index Scan en lugar de Seek
- Aviso en el plan: "Type conversion in expression (CONVERT_IMPLICIT...) may affect SeekPlan"
- Con intercalación SQL_ (por defecto en el contenedor) hay scan completo; con intercalación Windows, a veces "seek dinámico"
- Orígenes típicos: ORM que envían NVARCHAR, parámetros mal tipados, joins entre columnas de tipos distintos

En LegacyShop

usp_BuscarClientePorCodigo recibe @Codigo NVARCHAR(20) y compara con CodigoCliente VARCHAR(20). Resultado: 100.000 filas leídas para devolver una. Cambiar el tipo del parámetro lo convierte en un seek de 3 lecturas.

Cazar conversiones implícitas en la caché de planes

```
WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
SELECT TOP (20)
    DB_NAME(st.dbid) AS bd, qs.execution_count,
    qs.total_logical_reads / qs.execution_count AS lecturas_medias,
    w.value('@Expression', 'nvarchar(400)') AS conversion,
    st.text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
CROSS APPLY qp.query_plan.nodes('//Warnings/PlanAffectingConvert') AS t(w)
WHERE w.value('@ConvertIssue', 'nvarchar(50)') = 'Seek Plan'
ORDER BY qs.total_logical_reads DESC;
```

Consulta costosa sobre la caché: lanzadla fuera de hora punta o filtrando por base de datos.

Demo en vivo · La conversión implícita en acción

```
SET STATISTICS IO ON;

-- Como lo llama la aplicación (parámetro NVARCHAR)
EXEC dbo.usp_BuscarClientePorCodigo @Codigo = N'CLI-0004242';
-- Table 'Clientes'. Scan count 1, logical reads ≈ 1.000+

-- Misma búsqueda con el tipo correcto
DECLARE @c varchar(20) = 'CLI-0004242';
SELECT ClienteID, Nombre, Email FROM dbo.Clientes WHERE CodigoCliente = @c;
-- Table 'Clientes'. Scan count 0, logical reads 3-6

-- Al revés NO hay problema: la conversión cae sobre el literal
SELECT ProductoID FROM dbo.Productos WHERE Nombre = 'Taladro'; -- Nombre es NVARCHAR
```

Demostración del instructor (8 min). Mostrar el aviso en el operador SELECT y el Index Scan.

Spills a tempdb

- Sort, Hash Match y Exchange (paralelismo) necesitan memoria; si no cabe, vuelcan a tempdb
- Aviso en el plan real: "Operator used tempdb to spill data during execution with spill level N"
- Nivel 1: un volcado; niveles altos: recursión del hash, muy caro
- Causa principal: subestimación de filas o de tamaño de fila (columnas VARCHAR anchas estimadas al 50 %)
- Visibles también en XE (sort_warning, hash_spill_details) y en Query Store (tempdb_space_used)

Antes de pedir más memoria

Un spill casi siempre es un síntoma de mala estimación. Arreglad la estimación o evitad la ordenación con un índice que ya entregue los datos ordenados. El feedback de memoria ayuda en consultas repetitivas.

Key Lookup y RID Lookup

Qué son

- Un índice no agrupado encuentra las filas, pero le faltan columnas
- Key Lookup: se busca cada fila en el índice agrupado
- RID Lookup: igual, pero sobre un montículo (heap)
- Siempre dentro de un Nested Loops, una búsqueda por fila
- Con pocas filas es barato; con miles, muy caro

Cómo se diagnostican

- Propiedad Number of Executions del lookup = filas del seek
- Output List: qué columnas faltan en el índice
- Tipping point: a partir de cierto % el optimizador prefiere un scan
- Si la estimación falla, elige lookup para 250.000 filas
- Solución de fondo: índice cubriente con INCLUDE (sesión 4)

Demo en vivo · El coste real de un lookup

```
SET STATISTICS IO, TIME ON;

-- Plan de búsqueda forzado para el cliente grande
SELECT PedidoID, FechaPedido, Total, Estado, Canal
FROM dbo.Pedidos WITH (INDEX (IX_Pedidos_ClienteID))
WHERE ClienteID = 1;
-- logical reads ≈ 750.000 (seek + 250.000 key lookups)

-- Scan de la tabla para el mismo cliente
SELECT PedidoID, FechaPedido, Total, Estado, Canal
FROM dbo.Pedidos WITH (INDEX (1))
WHERE ClienteID = 1;
-- logical reads ≈ 9.000-12.000 (lectura secuencial de todo el agrupado)
```

Las cifras dependen del tamaño de fila del contenedor; lo importante es el orden de magnitud.

Otros sospechosos habituales

- Funciones escalares T-SQL: se ejecutan una vez por fila, impiden paralelismo y ocultan su coste en el plan
- Scalar UDF inlining (2019+, compat 150) las convierte en expresiones... si cumplen los requisitos
- Nested Loops con millones de ejecuciones en el lado interno: estimación baja en el lado externo
- Table Spool / Index Spool "eager": el optimizador construye un índice temporal que os está pidiendo crear
- Predicados residuales, OR entre columnas distintas y LIKE con comodín inicial

Comprobar el inlining

```
SELECT OBJECT_NAME(object_id),  
is_inlineable FROM sys.sql_modules  
WHERE is_inlineable IS NOT NULL.  
GETDATE(), bucles WHILE o ciertas  
construcciones impiden la conversión.
```

Query Store para encontrar regresiones

- Informe "Regressed Queries": consultas cuyo rendimiento empeoró entre dos periodos
- Informe "Queries With High Variation": sospechosas de parameter sniffing
- Informe "Top Resource Consuming Queries": dónde se va la CPU, las lecturas o la duración
- Comparar planes de una misma consulta y forzar el bueno mientras se investiga
- Métricas de espera por consulta (sys.query_store_wait_stats) desde 2017

Forzar no es arreglar

Forzar un plan en Query Store es un torniquete: detiene la hemorragia, pero hay que revisarlo. Un cambio de esquema puede invalidarlo y aparecerá como forzado fallido en sys.query_store_plan.

Lab 5A · Leer planes y cazar conversiones implícitas

- Aplicar el método de cinco pasadas a tres planes de LegacyShop
- Reproducir el falso "en SSMS va rápido" con ARITHABORT
- Localizar la conversión implícita de usp_BuscarClientePorCodigo en la caché
- Corregirla y medir lecturas antes y después
- Identificar el operador donde nace un error de estimación

55 min

labs/
Lab05A_Planes_Conversiones
.md

Lab 5B · Spills, lookups y funciones escalares

- Provocar un spill en el informe de ventas por categoría y leer su nivel
- Relacionar el spill con la estimación de filas y de tamaño de fila
- Medir el coste real de los Key Lookups con el cliente 1
- Comprobar qué funciones escalares admiten inlining y medir la diferencia
- Detectar un Index Spool y proponer el índice que lo elimina

55 min

labs/
Lab05B_Spills_Lookups.md

Cierre de la sesión 3

- Un timeout es una decisión del cliente; "en SSMS va rápido" no demuestra nada
- Real frente a estimado: el primer operador con un error grande es la raíz
- Conversiones implícitas: el arreglo más barato y rentable que existe
- Spills y lookups son síntomas de estimación o de índices; no de falta de hardware
- Próxima sesión: tratamiento. Refactorización T-SQL, índices, hints y bloqueos

Para casa

Lanzad la consulta de conversiones implícitas en un entorno real y traed la que más lecturas acumule, anonimizada. Empezaremos la sesión 4 analizando una de las vuestras.